CSS Interview Preparation Guide (Fresher/Junior Handbook)

A **professional, interview-first** CSS handbook with tricky concepts, clear explanations, code examples, best practices, and interview-ready answers.

---

1) CSS Basics

Question: What is CSS and how does it work in the browser?

Short Interview Answer: CSS (Cascading Style Sheets) styles HTML. The browser parses HTML/CSS, builds the DOM/CSSOM, combines them into a render tree, then lays out and paints pixels.

Explanation: CSS is a declarative language for styling. The browser converts CSS into the **CSSOM**, then matches selectors to DOM elements. Final styles are computed via the cascade + specificity + inheritance.

Example Code:

```
body { font-family: system-ui; }
.card { border: 1px solid #e5e7eb; padding: 16px; }
```

Best Practices:

- Keep CSS modular and predictable.
- Prefer modern layout (Flexbox/Grid) over floats.

Question: What are the different ways to apply CSS?

Short Interview Answer: Inline (`style`), internal (`<style>` in head), external stylesheet (`<link>`). External is best for maintainability and caching.

Explanation: Inline styles have high specificity and hurt reuse. External stylesheets are cached and scale better.

Example Code:

```
<link rel="stylesheet" href="styles.css">
```

Best Practices:

- Prefer external CSS for production.
- Avoid inline styles except dynamic one-offs.

Question: What is the cascade in CSS?

Short Interview Answer: Cascade decides the winning styles based on importance (`!important`), origin (author/user-agent), specificity, and source order.

Explanation: When multiple rules target the same property, the browser picks the final one using cascade rules.

Example Code:

```
p { color: blue; }
.text { color: red; }  /* wins over element selector */
```

Best Practices:

- Avoid `!important` (use it only for utilities or overrides in limited scope).

---

2) Selectors

Question: Difference between class selector and ID selector?

Short Interview Answer: Class (`.btn`) can be reused; ID (`#header`) should be unique. IDs have higher specificity than classes.

Explanation: IDs can cause overly strong selectors and harder overrides.

Example Code:

```
.btn { padding: 8px 12px; }
#header { position: sticky; }
```

Best Practices:

- Prefer classes for styling; use IDs for anchors/JS, not styling.

Question: What are combinators in CSS?

Short Interview Answer: Combinators define relationships: descendant (`A B`), child (`A > B`), adjacent sibling (`A + B`), general sibling (`A ~ B`).

Explanation: They control how selectors match DOM structure.

Example Code:

```
.card p { margin: 0; }      /* descendant */
.card > p { margin: 0; }    /* direct child */
label + input { margin-top: 6px; } /* adjacent sibling */
```

Best Practices:

- Keep selectors shallow; avoid overly specific DOM-dependent selectors.

Question: What is `:not()` and why is it useful?

Short Interview Answer: `:not()` excludes elements that match a selector.

Explanation: It improves readability for “all except X” rules.

Example Code:

```
button:not(.primary) { opacity: 0.8; }
```

Best Practices:

- Don’t overuse complex selectors that hurt maintainability.

---

3) Box Model

Question: What is the CSS Box Model?

Short Interview Answer: It describes element sizing: **content + padding + border + margin**.

Explanation: Actual space used depends on `box-sizing`. Default is `content-box` where width/height apply to content only.

Example Code:

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

Best Practices:

- Use `box-sizing: border-box;` globally to reduce sizing surprises.

Question: Difference between `content-box` and `border-box`?

Short Interview Answer: `content-box`: width excludes padding/border. `border-box`: width includes padding/border.

Explanation: With `border-box`, layout is simpler because final size stays fixed.

Example Code:

```
* { box-sizing: border-box; }
```

Best Practices:

- Apply `border-box` to all elements including pseudo elements.

Question: What is margin collapsing?

Short Interview Answer: Vertical margins of block elements can collapse into a single margin (largest one) under certain conditions.

Explanation: Common between parent/first child or adjacent blocks (not for flex/grid items).

Example Code:

```
h1 { margin-bottom: 20px; }
p  { margin-top: 10px; } /* may collapse to 20px total */
```

Best Practices:

- Use padding/border to prevent collapsing when needed.
- Prefer consistent spacing utilities.

---

4) Display Properties

Question: Difference between `display: block`, `inline`, and `inline-block`?

Short Interview Answer: `block` starts new line and can set width/height; `inline` flows in text and ignores width/height; `inline-block` flows inline but supports width/height.

Explanation: Display affects layout participation and sizing.

Example Code:

```
.badge { display: inline-block; padding: 4px 8px; }
```

Best Practices:

- Use Flexbox/Grid for layout rather than display hacks.

Question: Difference between `display: none` and `visibility: hidden`?

Short Interview Answer: `display:none` removes from layout; `visibility:hidden` keeps space but hides visually.

Explanation: `display:none` also removes from accessibility tree in most cases.

Example Code:

```
.hidden { display: none; }
.invisible { visibility: hidden; }
```

Best Practices:

- For accessible hiding, prefer specific patterns (e.g., visually-hidden utility).

---

5) Positioning

Question: Explain `position: relative/absolute/fixed/sticky`.

Short Interview Answer:

- `relative`: stays in normal flow; offsets move it visually.
- `absolute`: removed from flow; positioned relative to nearest positioned ancestor.
- `fixed`: relative to viewport.
- `sticky`: toggles between relative and fixed within scroll container.

Explanation: Absolute positioning uses the nearest ancestor with `position` not `static`.

Example Code:

```
.container { position: relative; }
.tooltip { position: absolute; top: 100%; left: 0; }
```

Best Practices:

- Use positioning for overlays/tooltips; use Flex/Grid for layout.

Question: What is containing block for absolutely positioned elements?

Short Interview Answer: Usually the nearest ancestor with `position` not `static`; otherwise the initial containing block.

Explanation: Also can be affected by transforms/filters in modern browsers.

Example Code:

```
.parent { position: relative; }
.child { position: absolute; inset: 0; }
```

Best Practices:

- Explicitly set `position: relative` on the intended container.

---

6) Flexbox

Question: What is Flexbox and when do you use it?

Short Interview Answer: A 1D layout system for arranging items in a row or column, great for alignment and distribution.

Explanation: Flex container controls main/cross axis. Items can grow/shrink.

Example Code:

```
.row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
}
```

Best Practices:

- Use `gap` instead of margins between items.

Question: Explain `flex: 1`, `flex-grow`, `flex-shrink`, `flex-basis`.

Short Interview Answer: `flex` is shorthand: `flex: grow shrink basis`. `flex:1` means grow=1, shrink=1, basis=0.

Explanation: Basis defines initial size before distributing free space.

Example Code:

```
.sidebar { flex: 0 0 260px; }
.content { flex: 1 1 auto; }
```

Best Practices:

- Prefer explicit basis for sidebars.

Question: Difference between `justify-content` and `align-items`?

Short Interview Answer: `justify-content` aligns along main axis; `align-items` along cross axis.

Explanation: Main axis depends on `flex-direction`.

Example Code:

```
.flex { display:flex; flex-direction: column; justify-content: center; a
```

Best Practices:

- Always state axis in your explanation in interviews.

---

7) CSS Grid

Question: What is CSS Grid and when should you use it?

Short Interview Answer: Grid is a 2D layout system for rows and columns; best for page-level layouts and complex alignments.

Explanation: Grid defines tracks and places items precisely.

Example Code:

```
.layout {
  display: grid;
  grid-template-columns: 260px 1fr;
  gap: 16px;
}
```

Best Practices:

- Use Grid for overall layout, Flexbox for components.

Question: Explain `fr` unit in Grid.

Short Interview Answer: `fr` is a fraction of available free space in the grid container.

Explanation: Tracks with `fr` share remaining space.

Example Code:

```
.grid { display:grid; grid-template-columns: 1fr 2fr; }
```

Best Practices:

- Combine `minmax()` with `fr` for responsive tracks.

Question: What is `minmax()` and why is it useful?

Short Interview Answer: It sets min and max size for a grid track.

Explanation: Great for responsive columns.

Example Code:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
```

Best Practices:

- Use `auto-fit/auto-fill` patterns for responsive cards.

---

8) Responsive Design

Question: What is responsive design?

Short Interview Answer: Building layouts that adapt to different screen sizes using fluid grids, flexible media, and media queries.

Explanation: Responsive design avoids fixed widths and uses relative units and breakpoints.

Example Code:

```
.container { width: min(1100px, 100% - 32px); margin: 0 auto; }
img { max-width: 100%; height: auto; }
```

Best Practices:

- Prefer mobile-first CSS.

Question: What is mobile-first approach?

Short Interview Answer: Write base styles for small screens, then add enhancements with `min-width` media queries.

Explanation: This often reduces overrides and improves performance.

Example Code:

```
.card { padding: 12px; }
@media (min-width: 768px) { .card { padding: 20px; } }
```

Best Practices:

- Use `min-width` for progressive enhancement.

---

9) Media Queries

Question: How do media queries work?

Short Interview Answer: They apply CSS rules conditionally based on device/viewport features like width, prefers-color-scheme, orientation.

Explanation: Browser evaluates conditions; matching rules join the cascade.

Example Code:

```
@media (max-width: 600px) {
  .nav { display: none; }
}
@media (prefers-color-scheme: dark) {
  body { background: #0b1020; color: #e5e7eb; }
}
```

Best Practices:

- Use meaningful breakpoints based on design, not device names.

---

10) CSS Units

Question: Difference between `px`, `%`, `em`, `rem`, `vw`, `vh`?

Short Interview Answer:

- `px` : fixed pixel-like unit
- `%` : relative to parent/reference
- `em` : relative to element font-size
- `rem` : relative to root font-size
- `vw/vh` : relative to viewport size

Explanation: `rem` is predictable for typography scale; `em` is useful for component-relative sizing.

Example Code:

```
html { font-size: 16px; }
h1 { font-size: 2rem; }  /* 32px */
.card { padding: 1.25rem; }
```

Best Practices:

- Prefer `rem` for typography and spacing scale.
- Be careful with `vh` on mobile (browser UI affects it).

Question: What is `clamp()` and why is it used?

Short Interview Answer: `clamp(min, preferred, max)` creates fluid values bounded by min and max.

Explanation: Useful for responsive font sizes without many breakpoints.

Example Code:

```
h1 { font-size: clamp(1.5rem, 2vw + 1rem, 3rem); }
```

Best Practices:

- Use clamp for fluid typography.

---

11) Pseudo Classes & Pseudo Elements

Question: Difference between pseudo-class and pseudo-element?

Short Interview Answer: Pseudo-classes select a state (`:hover` , `:focus`), pseudo-elements target part of an element (`::before` , `::first-line`).

Explanation: Pseudo-elements create or style virtual subparts.

Example Code:

```
a:hover { text-decoration: underline; }
.badge::before { content: "★ "; color: gold; }
```

Best Practices:

- Ensure focus styles for keyboard users (`:focus-visible`).

Question: What is `:nth-child()` and common mistakes?

Short Interview Answer: It selects elements based on their position among siblings; common mistake is confusing it with `:nth-of-type()` .

Explanation: `:nth-child(2)` selects the 2nd child regardless of tag.

Example Code:

```
li:nth-child(odd) { background: #f3f4f6; }
/* vs */
p:nth-of-type(2) { color: red; }
```

Best Practices:

- Use `:nth-of-type` when you mean “2nd paragraph”.

---

12) Specificity & Cascade

Question: How is specificity calculated?

Short Interview Answer: Specificity order: inline styles > IDs > classes/attributes/pseudo-classes > elements/pseudo-elements.

Explanation: More specific selectors win when importance and origin are same.

Example Code:

```
#app .btn { color: red; }   /* stronger */
.btn { color: blue; }
```

Best Practices:

- Keep selectors low-specificity; avoid chaining many selectors.

Question: What does `!important` do and when (if ever) to use it?

Short Interview Answer: It boosts priority in cascade, overriding normal specificity. Use sparingly for utilities or unavoidable third-party overrides.

Explanation: Too much `!important` creates specificity wars.

Example Code:

```
.u-hidden { display: none !important; }
```

Best Practices:

- Prefer better architecture (utilities/layers) over `!important`.

---

13) Z-Index & Stacking Context

Question: When does `z-index` work?

Short Interview Answer: `z-index` works on positioned elements (`position` not `static`) or flex/grid items in stacking contexts.

Explanation: Stacking context controls painting order. A child can't escape parent stacking context.

Example Code:

```
.modal { position: fixed; z-index: 1000; }
.backdrop { position: fixed; z-index: 900; }
```

Best Practices:

- Maintain a z-index scale (tokens) to avoid random huge values.

Question: What creates a stacking context?

Short Interview Answer: Examples: `position` + `z-index`, `transform`, `opacity < 1`, `filter`, `isolation: isolate`, `will-change`.

Explanation: Once created, children stack inside it.

Example Code:

```
.card { transform: translateZ(0); } /* creates stacking context */
```

Best Practices:

- Avoid unnecessary transforms on containers if you rely on z-index.

---

14) Transitions & Animations

Question: What is the difference between transitions and animations?

Short Interview Answer: Transitions animate between two states; animations define keyframes over time.

Explanation: Transitions require a property change. Animations can loop and run independently.

Example Code:

```
.btn { transition: transform 150ms ease; }
.btn:hover { transform: translateY(-1px); }

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.03); }
  100% { transform: scale(1); }
}
.loader { animation: pulse 1s infinite; }
```

Best Practices:

- Prefer animating `transform` and `opacity` for performance.

Question: How do you respect reduced motion preferences?

Short Interview Answer: Use `@media (prefers-reduced-motion: reduce)` to reduce/disable animations.

Explanation: Some users experience motion sickness.

Example Code:

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

Best Practices:

- Keep animations subtle and optional.

---

15) CSS Variables

Question: What are CSS variables and how do they differ from preprocessor variables?

Short Interview Answer: CSS variables (`--x`) are runtime, cascade-aware and can change dynamically; Sass variables are compile-time.

Explanation: Because they participate in cascade, you can theme components.

Example Code:

```
:root { --brand: #2563eb; }
.button { background: var(--brand); }
.dark { --brand: #22c55e; }
```

Best Practices:

- Use variables for design tokens (colors, spacing, z-index scale).

---

16) Performance Best Practices

Question: What makes CSS slow and how to optimize?

Short Interview Answer: Complex selectors, large stylesheets, heavy repaint/reflow triggers. Optimize by simplifying selectors, reducing unused CSS, and animating composite-friendly properties.

Explanation: Browsers match selectors right-to-left. Layout thrashing happens when JS forces sync layout.

Example Code:

```
/* Avoid */
.page .content .card .title span { ... }

/* Prefer */
.cardTitle { ... }
```

Best Practices:

- Use `content-visibility` for large offscreen content (carefully).
- Remove unused CSS (build tools).

17) Common Interview Traps

Question: Why doesn't `margin-top` work on an inline element?

Short Interview Answer: Vertical margins don't affect inline layout the same way; convert to `inline-block` or block/flex.

Explanation: Inline formatting context ignores some box properties.

Example Code:

```
.tag { display: inline-block; margin-top: 10px; }
```

Best Practices:

- Use appropriate display type.

Question: Why is `height: 100%` not working?

Short Interview Answer: Percent heights need a defined height on the parent chain.

Explanation: If parent height is `auto`, 100% has no reference.

Example Code:

```
html, body { height: 100%; }
.page { height: 100%; }
```

Best Practices:

- Prefer `min-height: 100vh` for full-viewport sections.

Question: Why is `z-index` not working?

Short Interview Answer: Because the element isn't positioned or it's trapped inside a stacking context.

Explanation: `z-index` doesn't apply to static positioned elements.

Example Code:

```
.box { position: relative; z-index: 2; }
```

Best Practices:

- Inspect stacking contexts in devtools.

---

18) Rapid Revision Cheat Sheet

- **Cascade order:** importance → origin → specificity → source order
- **Specificity:** inline > #id > .class/[attr]/:pseudo-class > element/::pseudo-element
- **Box model:** content + padding + border + margin
- **Use `border-box`:** predictable sizing
- **Flexbox:** 1D (row/column), `justify` main-axis, `align` cross-axis
- **Grid:** 2D, `fr`, `minmax`, `auto-fit`
- **Positioning:** relative/absolute/fixed/sticky
- **Responsive:** mobile-first, fluid units, breakpoints
- **Units:** `rem` for scale, `clamp()` for fluid sizing
- **Stacking:** z-index + stacking contexts
- **Perf:** animate `transform/opacity`, avoid deep selectors

Generated from markdown for sharing (PDF-friendly).